宁波大学信息学院

实验报告

实验课程名称： 汇编语言与接口设计

学　　院： 信息学院

专　　业： 计算机科学与技术

指 导 教 师： 陈瑜

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 22 级计算机科学与技术 2 班

学　　期： 2024-2025 第一学期

信息学院实验中心制

实验								
成绩								

实验项目名称	实验 3　8251 串行接口应用实验

一、　实验目的与要求：

1. 学习并掌握 **8251** 的工作方式及应用编程。

2. 了解有关串口通讯的知识。

3. 消化“自发自收实验”

4. 修改程序：

键盘输入数据 -> 发送数据到端口 -> 接受数据 -> 加 **1** 并显示

二、　实验设备：

Masm for windows 集成实验环境共享版

Windows11 OS

TD-PITE 实验装置一套

三、实验方法（原理、流程图）

实验原理：

1. 8251 的基本性能

8251 是可编程的串行通信接口，可以管理信号变化范围很大的串行数据通信。有下列基本性能:

(1)通过编程，可以工作在同步方式，也可以工作在异步方式。

(2)同步方式下，波特率为 **0~64K**，异步方式下，波特率为 **0~19.2K**。

(3)在同步方式时，可以用 **5~8** 位来代表字符，内部或外部同步，可自动插入同步字符。

(4)在异步方式时，也使用 **5~8** 位来代表字符，自动为每个数据增加 **1** 个启动位，并能够根据编程为每个数据增加 **1** 个、**1.5** 个或 **2** 个停止位。

(5)具有奇偶、溢出和帧错误检测能力。

(6)全双工，双缓冲器发送和接收器。

注意，**8251** 尽管通过了 **RS-232** 规定的基本控制信号，但并没有提供规定的全部信号。

2. 8251 的内部结构及外部引脚

8251 的内部结构图如图 **1** 所示，可以看出，**8251** 有 **7** 个主要部分，即数据总线缓冲器、读/写控制逻辑电路、调制/解调控制电路、发送缓冲器、发送控制电路、接收缓冲器和接收控制电路，图中还标识出了每个部分对外的引脚。

8251 的外部引脚如图 **2** 所示，共 **28** 个引脚，每个引脚信号的输入输出方式如图中的箭头方向所示。

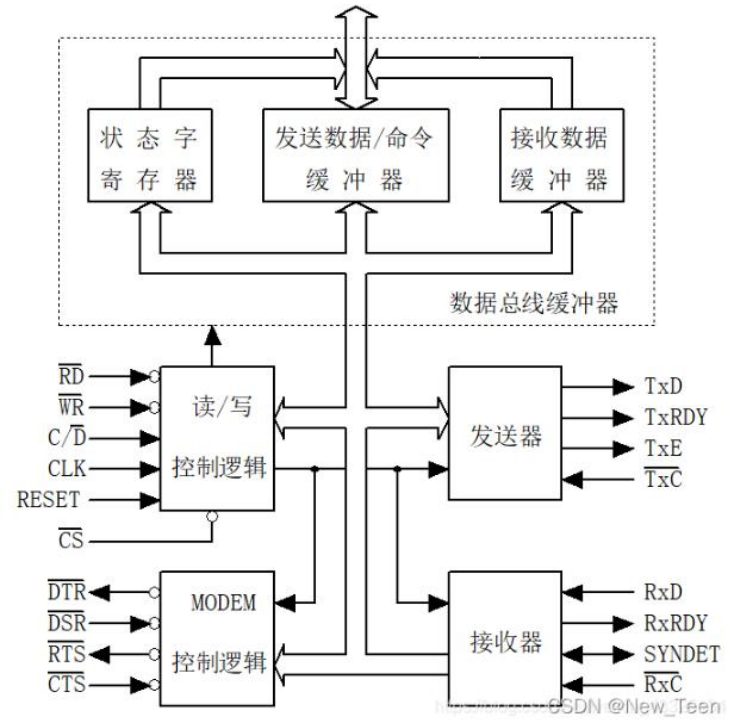


图 1 8251 **内部结构图**

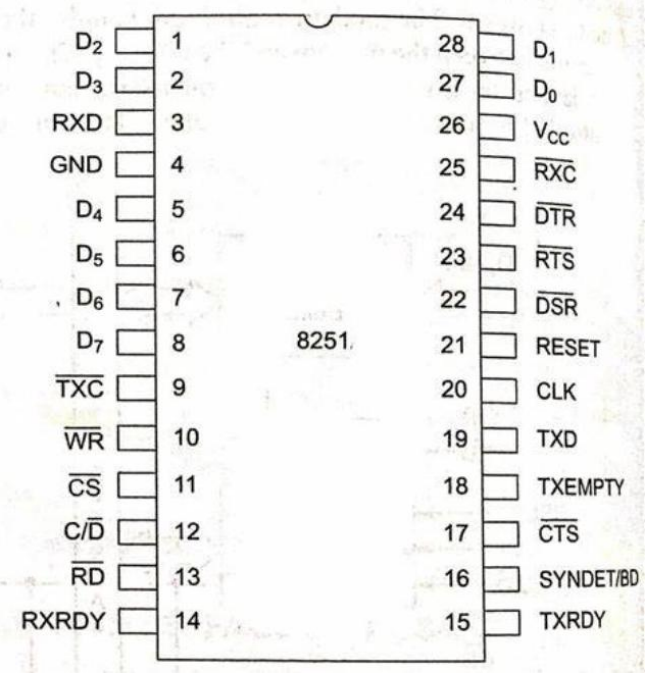


图 2 8251 **外部引脚图**

3. 8251 在异步方式下的 TXD 信号上的数据传输格式

图 **3** 示意了 **8251** 工作在异步方式下的 **TXD** 信号上的数据传输格式。数据位与停止位的位数可以由编程指定。

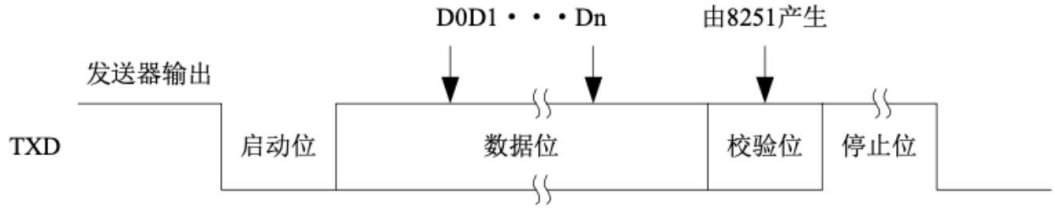


图 3 8251 **工作在异步方式下** TXD **信号的数据传输格式**

4. 8251 的编程

对 **8251** 的编程就是对 **8251** 的寄存器的操作,下面分别给出 **8251** 的几个寄存器的格式。

(1)方式控制字

方式控制字用来指定通信方式及其方式下的数据格式,具体各位的定义如图 4 所示。

D7	D6	D5	D4	D3	D2	D1	D0
SCS/S2	ESD/S1	EP	PEN	L2	L1	B2	B1
同步/停止位		奇偶校验		字符长度		波特率系数	
同步 (D1D0=00) X0=内同步 X1=外同步 0X=双同步 1X=单同步	异步 (D1D0≠0) 00=不用 01=1 位 10=1.5 位 11=2 位	X0=无校验 01=奇校验 11=偶校验		00=5 位 01=6 位 10=7 位 11=8 位		异步 00=不用 01=01 10=16 11=64	同步 00=同步 方式标志

图 4 8251 方式控制字

(2)命令控制字

命令控制字用于指定 **8251** 进行某种操作(如发送、接收、内部复位和检测同步字符等)或处于某种工作状态，以便接收或发送数据。图 **5** 所示的是 **8251** 命令控制字各位的定义。

D7	D6	D5	D4	D3	D2	D1	D0
EH	IR	RTS	ER	SBRK	RxE	DTR	TxEN
进入搜索 1=允许搜索	内部复位 1=使 8251 返回方式控制字	请求发送 1=使 RTS 输出 0	错误标志复位使错误标志 PE、OE、FE 复位	发中止字符 1=使 TXD 为低 0=正常工作	接收允许 1=允许 0=禁止	数据终端准备好 1=使DTR输出0	发送允许 1=允许 0=禁止

图 5 8251 命令控制字格式

(3)状态字

CPU 通过状态字来了解 **8251** 当前的工作状态，以决定下一步的操作，**8251** 的状态字如图 **6** 所示。

D7	D6	D5	D4	D3	D2	D1	D0
DSR	SYNDET	FE	OE	PE	TxE	RxRDY	TxRDY
数据装置就绪: 当 DSR 输入为 0 时，该位为 1	同步检测	帧错误：该标志仅用于异步方式，当在任一字符的结尾没有检测到有效的停止位时，该位置 1。此标志由命令控制字中的位 4 复位。	溢出错误：在下一个字符变为可用前，CPU 没有把字符读走，此标志置 1。此错误出现时上一字符已丢失。	奇偶错误: 当检测到奇偶错误时此位置 1。	发送器空	接收就绪为1表明接收到一个字符。	发送就绪为1表明发送缓冲器空。

图 6 8251 状态字格式

(4)系统初始化

8251 的初始化和操作流程如图 **7** 所示。

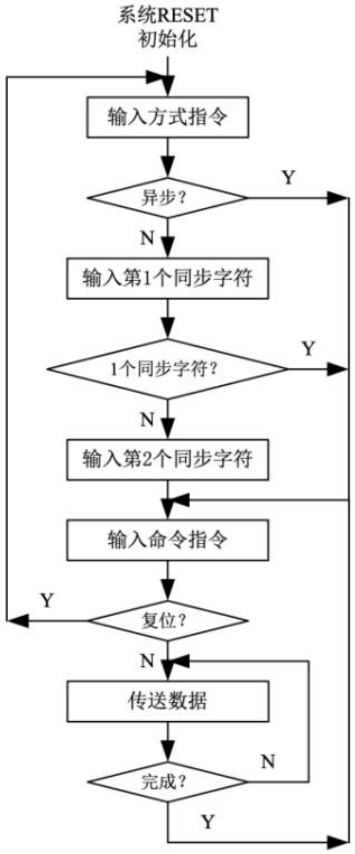


图 7 8251 初始化流程图

5. 8251 实验单元电路图

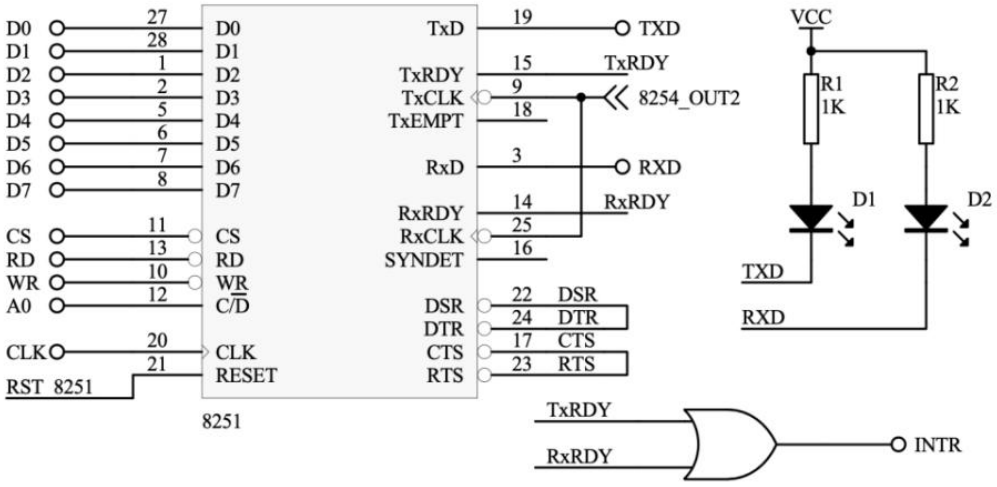


图 8 8251 实验单元电路图

四、实验过程、步骤及内容

1. 自发自收基础实验

通过自收自发实验，可以验证硬件及软件设计，常用于自测试。具体实验步骤如下：

(1) 参考实验接线图如图 9 所示，按图连接实验线路。

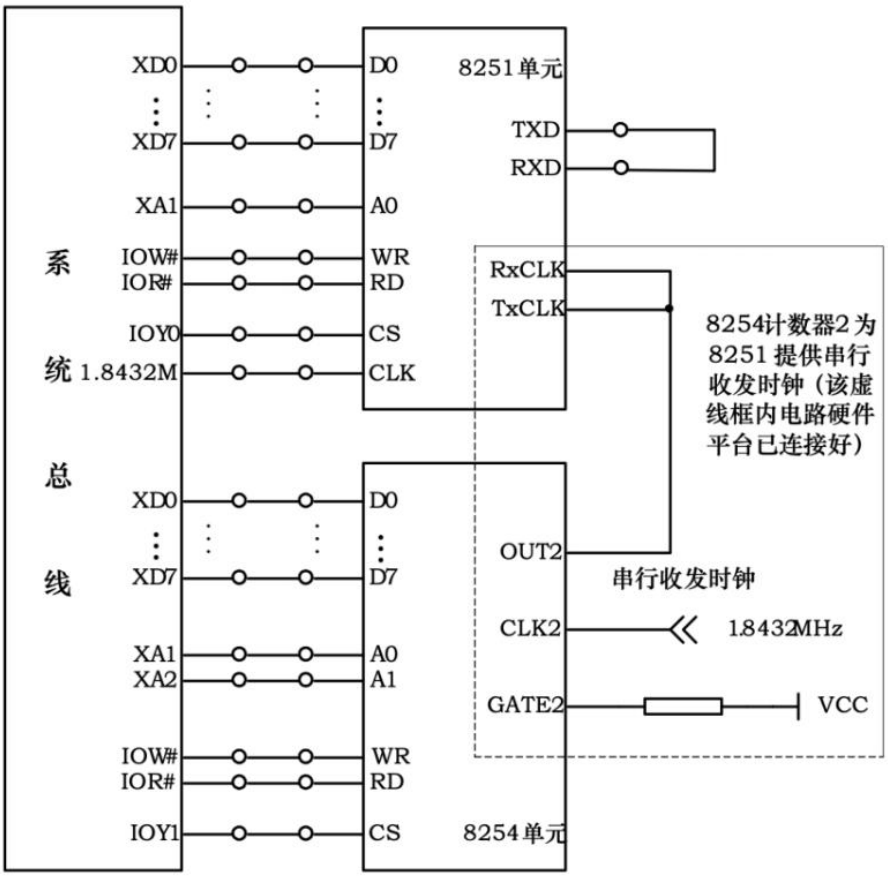


图 9 自收自发实验接线图

实验代码如下：

```
;=============================================================
; 文件名: A82512.ASM
; 功能描述: 自收自发实验程序, 源地址 4000H, 目的地址 3000H
;=============================================================

IOY0 EQU 0600H          ; IOY0 起始地址
```

```
7. IOY1 EQU 0640H          ; IOY1 起始地址
8. M8251_DATA EQU IOY0+00H*2
9. M8251_CON EQU IOY0+01H*2
10.M8254_2 EQU IOY1+02H*2
11.M8254_CON EQU IOY1+03H*2
12.
13.SSTACK SEGMENT STACK
14.DW 64 DUP(?)            ; 初始化栈区
15.SSTACK ENDS
16.
17.CODE SEGMENT
18.ASSUME CS:CODE
19.
20.START:
21.    MOV AX, 0000H
22.    MOV DS, AX          ; 初始化数据段
23.
24.    ; 初始化 8254，得到收发时钟
25.    MOV AL, 0B6H
26.    MOV DX, M8254_CON
27.    OUT DX, AL
28.    MOV AL, 0CH
29.    MOV DX, M8254_2
30.    OUT DX, AL
31.    MOV AL, 00H
32.    OUT DX, AL
33.
34.    ; 复位 8251
35.    CALL INIT
36.    CALL DALLY
37.
38.    ; 配置 8251 为方式字
39.    MOV AL, 7EH
40.    MOV DX, M8251_CON
41.    OUT DX, AL
42.    CALL DALLY
43.
44.    ; 配置 8251 控制字
45.    MOV AL, 34H
46.    OUT DX, AL
47.    CALL DALLY
48.
49.    ; 设置源地址（4000H）和目的地址（3000H）
```

```
    MOV DI, 3000H
    MOV SI, 4000H
    MOV CX, 000AH

A1:
    MOV AL, [SI]            ; 读取源地址数据
    MOV DX, M8251_CON
    MOV AL, 37H             ; 发送数据命令
    OUT DX, AL

    MOV DX, M8251_DATA
    OUT DX, AL              ; 发送数据

    ; 等待发送缓冲区空
    CALL WAIT_SEND_EMPTY

    ; 等待接收缓冲区满
    CALL WAIT_RECEIVE_FULL

    MOV DX, M8251_DATA
    IN AL, DX               ; 读取接收到的数据
    MOV [DI], AL            ; 存储到目的地址

    INC DI                  ; 移动目的地址指针
    INC SI                  ; 移动源地址指针

    LOOP A1                 ; 循环处理

    MOV AX, 4C00H
    INT 21H                 ; 程序终止

;-------------------------------------------------------------
; 复位 8251 子程序
INIT:
    MOV AL, 00H
    MOV DX, M8251_CON
    OUT DX, AL
    CALL DALLY
    OUT DX, AL
    CALL DALLY
    OUT DX, AL
    CALL DALLY
    MOV AL, 40H
```

```
93.     OUT DX, AL
94.     RET
95.
96.;-------------------------------------------------------------
97.; 等待发送缓冲区空的子程序
98.WAIT_SEND_EMPTY:
99.     MOV DX, M8251_CON
100.A2:
101.     IN AL, DX
102.     AND AL, 01H
103.     JZ A2
104.     RET
105.
106.;-------------------------------------------------------------
107.; 等待接收缓冲区满的子程序
108.WAIT_RECEIVE_FULL:
109.     MOV DX, M8251_CON
110.A3:
111.     IN AL, DX
112.     AND AL, 02H
113.     JZ A3
114.     RET
115.
116.;-------------------------------------------------------------
117.; 延时子程序
118.DALLY:
119.     PUSH CX
120.     MOV CX, 3000H
121.A5:
122.     LOOP A5
123.     POP CX
124.     RET
125.
126.CODE ENDS
127.END START
```

(3)使用 E 命令更改 4000H 起始的 10 个单元中的数据。

(4)运行实验程序，待程序运行停止。

(5)查看 3000H 起始的 10 个单元中的数据，与初始化的数据进行比较，验证程序功能。

初始化数据段

- 设置 DS 寄存器为 0000H，初始化数据段。

初始化 8254 定时器

- 配置 8254 定时器以产生收发时钟。
- 发送控制字 0B6H 配置定时器。
- 发送时钟设置命令 0CH。

复位 8251 USART

- 调用 INIT 子程序复位 8251，确保串口通信设备处于已知状态。

配置 8251 USART 为方式字

- 发送方式字 7EH 设置 8251 的工作模式。

配置 8251 控制字

- 发送控制字 34H 配置 8251 的操作模式。

设置源地址和目的地址

- 设置源地址 4000H（从源读取数据）。
- 设置目的地址 3000H（存储接收到的数据）。

循环传输数据

- 循环读取源地址的数据，并将其通过 8251 USART 发送出去。
- 每发送一个数据字节后，等待发送缓冲区空，确保数据已发送。
- 等待接收缓冲区满，确保接收到了数据。
- 将接收到的数据存储到目的地址。
- 循环进行多次数据传输（这里设置了传输 10 次）。

程序终止

- 使用 MOV AX, 4C00H 和 INT 21H 结束程序。

2. 自发自收扩展实验

修改程序：

键盘输入数据 -> 发送数据到端口 -> 接受数据 -> 加 1 并显示

实验代码如下：

```
;=============================================================
;  文件名:  A82512.ASM
;  功能描述:  自收自发实验程序,  源地址  4000H,  目的地址  3000H
;=============================================================

IOY0  EQU  0600H                ;  IOY0  起始地址
IOY1  EQU  0640H                ;  IOY1  起始地址
M8251_DATA  EQU  IOY0+00H*2
M8251_CON  EQU  IOY0+01H*2
M8254_2  EQU  IOY1+02H*2
M8254_CON  EQU  IOY1+03H*2

SSTACK  SEGMENT  STACK
DW  64  DUP(?)                  ;  初始化栈区
SSTACK  ENDS

CODE  SEGMENT
ASSUME  CS:CODE

START:
        MOV  AX,  0000H
        MOV  DS,  AX                    ;  初始化数据段

        ;  初始化  8254，得到收发时钟
        MOV  AL,  0B6H
        MOV  DX,  M8254_CON
        OUT  DX,  AL
        MOV  AL,  0CH
        MOV  DX,  M8254_2
        OUT  DX,  AL
        MOV  AL,  00H
        OUT  DX,  AL

        ;  复位  8251
        CALL  INIT
        CALL  DALLY

        ;  配置  8251  为方式字
        MOV  AL,  7EH
```

```
40.             MOV   DX,   M8251_CON
41.             OUT   DX,   AL
42.             CALL   DALLY
43.
44.             ;  配置  8251  控制字
45.             MOV   AL,   34H
46.             OUT   DX,   AL
47.             CALL   DALLY
48.
49.             MOV   CX,   000AH                    ;  输入  10  个字符
50.
51.   A1:
52.             MOV   AH,   01H
53.             INT   21H                              ;  从键盘输入字符（存储
    到  AL）
54.             PUSH   AX                              ;  保存  AL  的值（输入字
    符）
55.
56.             MOV   AL,   37H
57.             MOV   DX,   M8251_CON
58.             OUT   DX,   AL                       ;  发送命令到  8251
59.
60.             POP   AX                               ;  恢复  AL（输入字符）
61.             MOV   DX,   M8251_DATA
62.             OUT   DX,   AL                       ;  发送数据到  8251
63.
64.             ;  等待发送缓冲区空
65.             CALL   WAIT_SEND_EMPTY
66.
67.             ;  等待接收缓冲区满
68.             CALL   WAIT_RECEIVE_FULL
69.
70.             MOV   DX,   M8251_DATA
71.             IN   AL,   DX                          ;  从  8251  读取接收到的
    数据
72.
73.             CMP   AL,   39H                      ;  判断是否接收到  '9'  字
    符
74.             JZ   A6                                ;  如果是  '9'，跳转
    到  A6
75.
76.             ;  加  1  并显示
77.   A4:
```

```
78.             INC   AL                                  ;  数据加  1
79.             MOV   DL,   AL
80.             MOV   AH,   02H
81.             INT   21H                                 ;  打印字符
82.             MOV   DL,   20H
83.             MOV   AH,   02H
84.             INT   21H                                 ;  打印空格
85.
86.             LOOP   A1                                 ;  循环输入
87.
88.             MOV   AX,   4C00H
89.             INT   21H                                 ;  程序终止
90.
91.   A6:
92.             ADD   AL,   07H                       ;  '9'  加  7，得到  'F'
93.             JMP   A4                                   ;  继续加  1  并显示
94.
95.   ;-------------------------------------------------------------
96.   ;  复位  8251  子程序
97.   INIT:
98.             MOV   AL,   00H
99.             MOV   DX,   M8251_CON
100.            OUT   DX,   AL
101.            CALL   DALLY
102.            OUT   DX,   AL
103.            CALL   DALLY
104.            OUT   DX,   AL
105.            CALL   DALLY
106.            MOV   AL,   40H
107.            OUT   DX,   AL
108.            RET
109.
110.  ;-------------------------------------------------------------
111.  ;  等待发送缓冲区空的子程序
112.  WAIT_SEND_EMPTY:
113.            MOV   DX,   M8251_CON
114.  A2:
115.            IN   AL,   DX
116.            AND   AL,   01H
117.            JZ   A2
118.            RET
119.
120.  ;-------------------------------------------------------------
```

```
;  等待接收缓冲区满的子程序
WAIT_RECEIVE_FULL:
        MOV  DX,  M8251_CON
A3:
        IN  AL,  DX
        AND  AL,  02H
        JZ  A3
        RET

;-------------------------------------------------------
;  延时子程序
DALLY:
        PUSH  CX
        MOV  CX,  3000H
A5:
        LOOP  A5
        POP  CX
        RET

CODE  ENDS
END  START
```

初始化：

- 设置数据段：MOV DS, AX。
- 初始化 8254 计数器以获得发送/接收时钟信号。
- 配置 8251 进行串口通信。

复位和配置 8251：

- 复位 8251，并设置其工作模式，配置为通信所需的方式字和控制字。

数据输入：

- 使用 DOS 中断 INT 21H 进行键盘输入，将用户输入的字符存储到 AL 寄存器中。

数据发送：

- 通过端口 M8251_CON 向 8251 发送控制命令（比如发送命令）。
- 将输入的字符传送到 8251 数据寄存器中发送。

等待发送缓冲区空：

- 程序进入一个循环,等待 8251 的发送缓冲区清空。只有当缓冲区为空时，才能发送新的数据。

等待接收缓冲区满：

- 程序进入另一个循环，等待 8251 接收到数据。当接收缓冲区满时，程序读取接收到的数据。

处理接收到的数据：

- 如果接收到的数据是 '9'（ASCII 39H），程序会跳到 A6 标签，进行特殊处理。
- 否则，将接收到的数据加 1，然后输出加 1 后的结果。

数据输出：

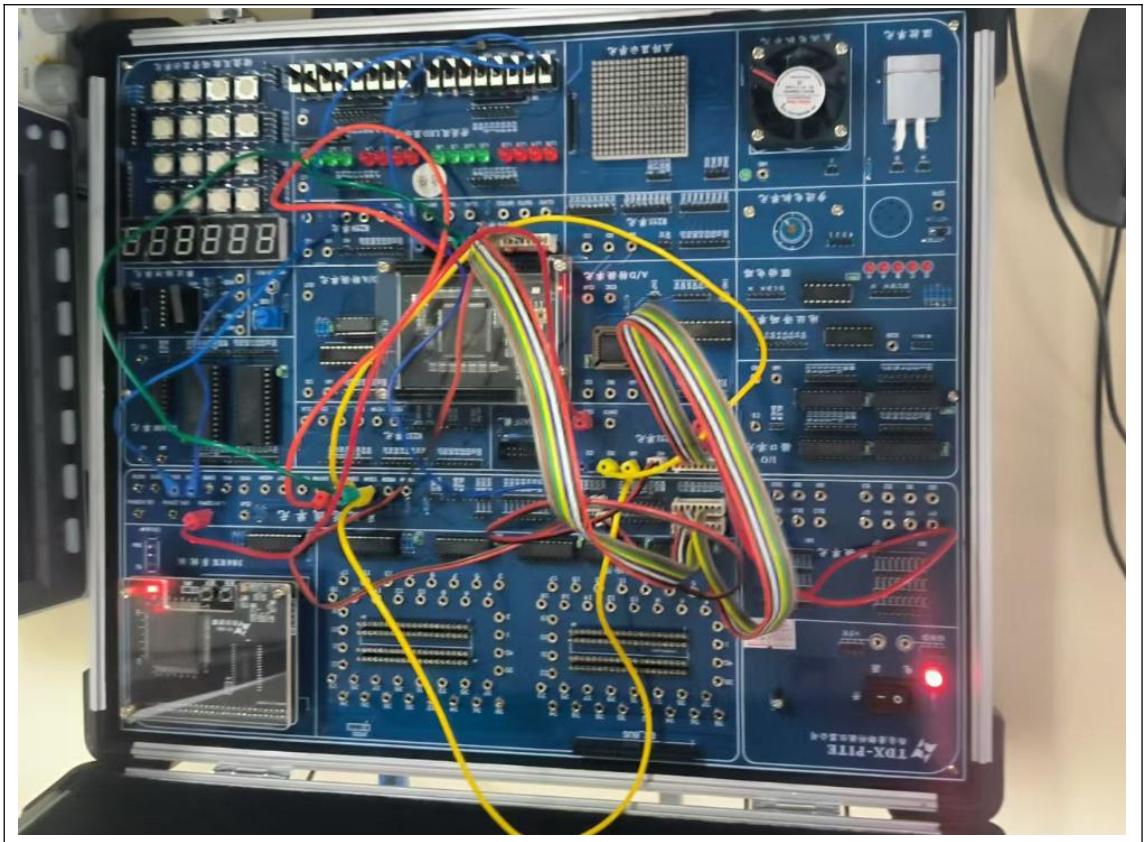
- 输出加 1 后的字符。
- 每输出一个字符后，输出一个空格。

程序结束：

- 程序通过 INT 21H 中断调用退出。

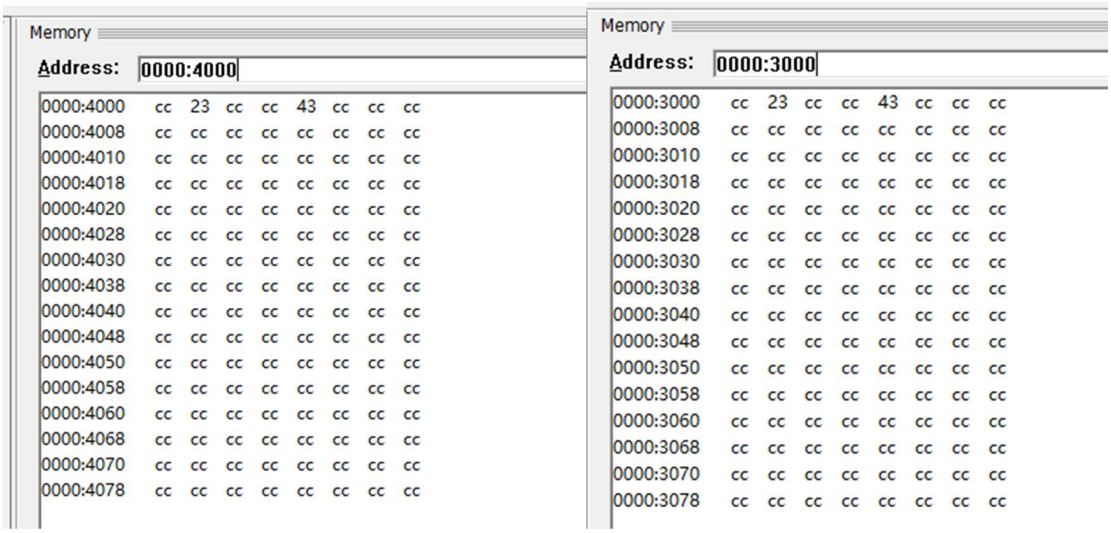
五、实验数据（现象）处理分析

实验连线图：



1. 自发自收基础实验

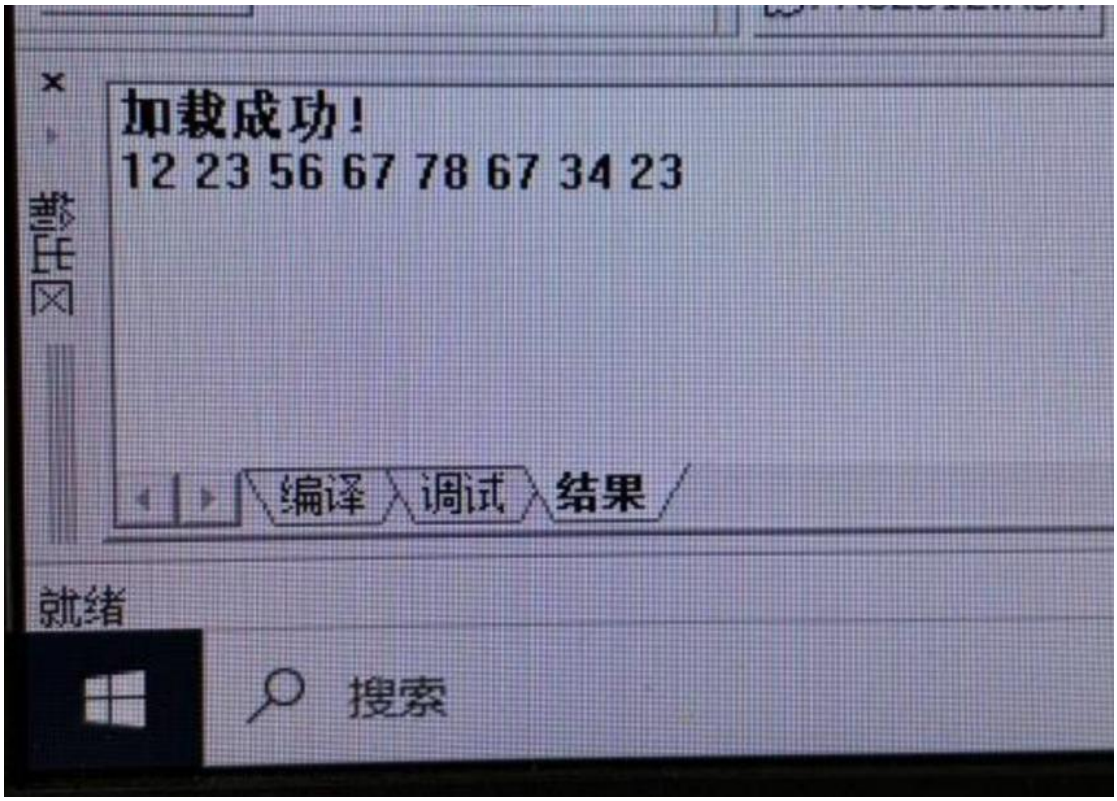
可以看到下图中 3000H 和 4000H 中的存储的数据相同，说明数据顺利从 4000H 发送到 3000H，符合实验要求。



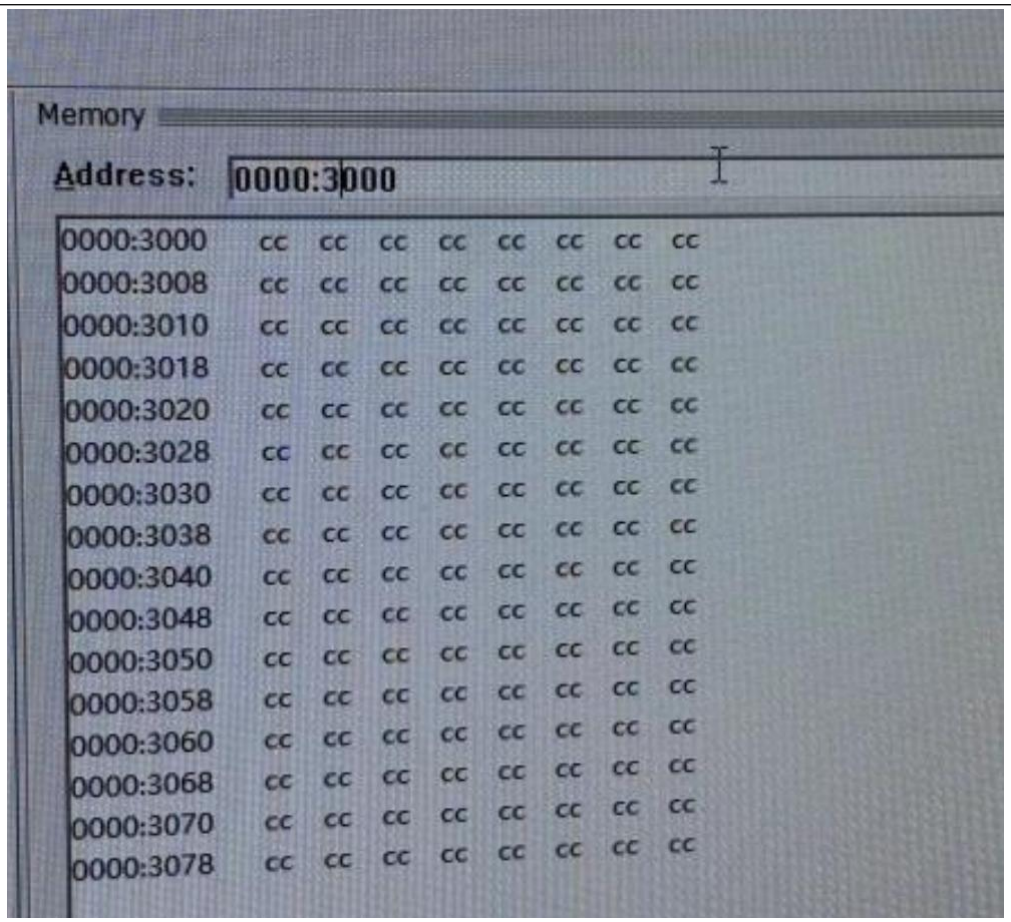
2. 自发自收扩展实验

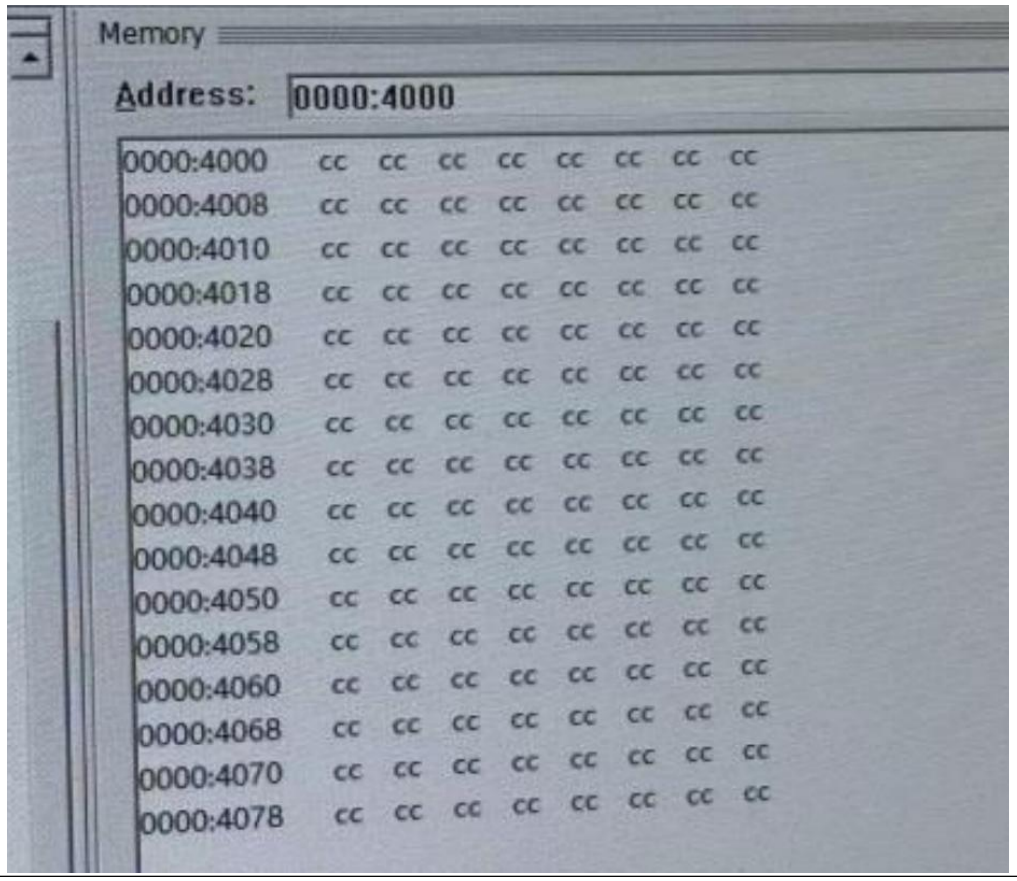
如下图所示，输入 1 时，在输出 1 后自动输出 2，输入 2 时，在输出 2

后自动输出 **3**，以此类推，符合实验需求。



可以看到下图中 **3000H** 和 **4000H** 中的存储的数据相同，说明数据顺利从 **4000H** 发送到 **3000H**，符合实验要求。


Memory
Address: 0000:3000
0000:3000 cc cc cc cc cc cc cc cc
0000:3008 cc cc cc cc cc cc cc cc
0000:3010 cc cc cc cc cc cc cc cc
0000:3018 cc cc cc cc cc cc cc cc
0000:3020 cc cc cc cc cc cc cc cc
0000:3028 cc cc cc cc cc cc cc cc
0000:3030 cc cc cc cc cc cc cc cc
0000:3038 cc cc cc cc cc cc cc cc
0000:3040 cc cc cc cc cc cc cc cc
0000:3048 cc cc cc cc cc cc cc cc
0000:3050 cc cc cc cc cc cc cc cc
0000:3058 cc cc cc cc cc cc cc cc
0000:3060 cc cc cc cc cc cc cc cc
0000:3068 cc cc cc cc cc cc cc cc
0000:3070 cc cc cc cc cc cc cc cc
0000:3078 cc cc cc cc cc cc cc cc


Memory
Address: 0000:4000
0000:4000 cc cc cc cc cc cc cc cc
0000:4008 cc cc cc cc cc cc cc cc
0000:4010 cc cc cc cc cc cc cc cc
0000:4018 cc cc cc cc cc cc cc cc
0000:4020 cc cc cc cc cc cc cc cc
0000:4028 cc cc cc cc cc cc cc cc
0000:4030 cc cc cc cc cc cc cc cc
0000:4038 cc cc cc cc cc cc cc cc
0000:4040 cc cc cc cc cc cc cc cc
0000:4048 cc cc cc cc cc cc cc cc
0000:4050 cc cc cc cc cc cc cc cc
0000:4058 cc cc cc cc cc cc cc cc
0000:4060 cc cc cc cc cc cc cc cc
0000:4068 cc cc cc cc cc cc cc cc
0000:4070 cc cc cc cc cc cc cc cc
0000:4078 cc cc cc cc cc cc cc cc

六、实验结论：

在这次实验中，我主要完成了 **8251** 串行接口的应用实验。通过异步方式，我成功地实现了从键盘输入数据并发送到端口，同时接收数据并对其进行加 **1** 操作后显示，完全符合实验要求。同时，我对原有程序进行了修改，扩展了自收自发实验的功能，使得整个过程能够自动进行数据的发送、接收和处理，这验证了 **8251** 串行接口的灵活性和可编程性。

在分析实验数据时，我发现存储在 **3000H** 和 **4000H** 地址的数据一致性良好，证明了数据在发送和接收过程中没有丢失或出错，验证了程序的正确性和稳定性。

通过这次实验，我不仅掌握了 **8251** 串行接口在异步模式下的工作原理和编程方法，还成功实现了数据的发送、接收和处理。实验结果符合预期，程序运行正常，达到了实验目标。这次实验让我对汇编语言和接口技术有了更深入的理解，也收获了不少经验。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。